

Hyper universalServiceId Mapping

How Carland maps partner service ids to our maintenance catalogue

Audience: Product Owner / stakeholders. Source of truth in code: com.carland.carland_service.enums.HyperServiceMapping

1. Short answer

Yes - we have a mapping service-side. It lives in the HyperServiceMapping enum. Hyper (and webhook ingest) sends a string field called universalServiceId on each service line. Carland does NOT convert that string into a numeric database id. Instead, it matches the string to our catalogue English name (services.name_en / Percentage.serviceNameEn).

Example: Hyper sends universalServiceId = "Engine oil & filter". That maps to the catalogue row whose name_en is also "Engine oil & filter" (enum constant ENGINE_OIL).

2. Why this exists

- Partner visits arrive with Hyper's own service identifiers (universalServiceId).
- Our app tracks maintenance progress per car using Percentage / services rows keyed by name_en.
- The mapping connects partner visit lines to the correct maintenance item so last/next km and dates can be synced for the driver.

3. How the logic works

3.1 What is stored

On ingest, Carland stores the Hyper universalServiceId mostly as-is on VisitServiceLine (raw partner value, with light normalization such as trimming / treating "other" as empty). The mapping enum is used later when syncing partner history onto Percentage rows.

3.2 Matching rules

- Primary key in the enum = canonical DB name_en (exact English catalogue name).
- Optional extraHyperIds = alternate Hyper spellings for the same catalogue row (example: Tyres vs Tires).
 - matches(hyperUniversalServiceId, percentageNameEn) returns true when the Hyper id belongs to that catalogue name (case-insensitive).
- toNameEn(hyperUniversalServiceId) resolves a Hyper id to name_en when unambiguous; returns empty if unknown or ambiguous.
- Unmapped values (for example "other") are skipped silently - no percentage update.

3.3 Where it is used in the product flow

HyperPercentageSyncService walks the car's Visit / VisitServiceLine history and, for each Percentage row, finds the latest partner line whose universalServiceId matches that row's serviceNameEn via HyperServiceMapping.matches(...). That match drives last/next service km and dates shown for the car.

4. Mapping table (current enum)

Hyper universalServiceId (and aliases) -> our services.name_en:

Hyper universalServiceId / aliases	Our name_en
Air filter	Air filter
Battery	Battery
Brake fluid	Brake fluid
Brake pads	Brake pads
Cabin filter	Cabin filter
Coolant (antifreeze)	Coolant (antifreeze)
Engine oil & filter	Engine oil & filter

Fuel filter	Fuel filter
Gas filter	Gas filter
Gas injectors	Gas injectors
Glow plugs	Glow plugs
HV battery / power-electronics coolant	HV battery / power-electronics coolant
Inverter Coolant (antifreeze)	Inverter Coolant (antifreeze)
Power steering fluid	Power steering fluid
Reduction-gear oil	Reduction-gear oil
Spark plugs	Spark plugs
Timing belt	Timing belt
Transmission fluid	Transmission fluid
Tyres (alias: Tires)	Tyres
Vaporiser service	Vaporiser service
Wheel alignment	Wheel alignment
Wheel balancing & rotation	Wheel balancing & rotation
Wheel balancing&rotation (alias of balancing)	Wheel balancing & rotation

5. Worked example for PO

Partner payload service line includes: universalServiceId = "Engine oil & filter".

- Enum entry: ENGINE_OIL("Engine oil & filter")
- Resolved catalogue name_en: Engine oil & filter
- If the car has a Percentage / service row with serviceNameEn = "Engine oil & filter", sync will attach this visit line's last/next mileage and dates to that maintenance item.
- There is no separate numeric "convert to id" step in this mapping layer - the bridge is the English service name used in our catalogue.

6. Important product notes

- This mapping is Hyper-oriented today. Adding another partner with different ids would require extending or generalizing this table.
- If Hyper sends a new universalServiceId that is not listed, it will not update any Percentage until engineering adds a mapping (or alias).
- Alias support exists for spelling variants (Tyres/Tires, compact wheel-balancing spelling).

7. Source code (as in repository)

File: src/main/java/com/carland/carland_service/enums/HyperServiceMapping.java

```
package com.carland.carland_service.enums;

import java.util.Optional;

/**
 * tr: Hyper'?n universalServiceId de?erlerini veritaban?ndaki servis adlar?na (services.name_en) e?leyen
 * enum.
 * en: Enum mapping Hyper's universalServiceId values to service names in the database (services.name_en).
 */
/**
 * Maps Hyper's raw {@code universalServiceId} to {@code services.name_en}.
 *
 * <p>Sync matches {@code Percentage.serviceNameEn} via {@link #matches(String, String)}.</p>
 *
 * <p>Primary value = exact DB {@code name_en}. {@code extraHyperIds} = alternate Hyper id spellings
 * for the same service row. Unmapped values (e.g. {@code "other"}) are skipped silently.</p>
 */
public enum HyperServiceMapping {

    AIR_FILTER("Air filter"),
    BATTERY("Battery"),
    BRAKE_FLUID("Brake fluid"),
    BRAKE_PADS("Brake pads"),
    CABIN_FILTER("Cabin filter"),
    COOLANT("Coolant (antifreeze)"),
    ENGINE_OIL("Engine oil & filter"),
    FUEL_FILTER("Fuel filter"),
    GAS_FILTER("Gas filter"),
    GAS_INJECTORS("Gas injectors"),
    GLOW_PLUGS("Glow plugs"),
    HV_BATTERY_COOLANT("HV battery / power-electronics coolant"),
    INVERTER_COOLANT("Inverter Coolant (antifreeze)"),
    POWER_STEERING_FLUID("Power steering fluid"),
    REDUCTION_GEAR_OIL("Reduction-gear oil"),
    SPARK_PLUGS("Spark plugs"),
    TIMING_BELT("Timing belt"),
    TRANSMISSION_FLUID("Transmission fluid"),
    /** DB {@code name_en} is {@code Tyres}; Hyper may send {@code Tyres} or {@code Tires}. */
    TYRES("Tyres", "Tires"),
    VAPORISER_SERVICE("Vaporiser service"),
    WHEEL_ALIGNMENT("Wheel alignment"),
    WHEEL_BALANCING("Wheel balancing & rotation"),
    WHEEL_BALANCING_COMPACT("Wheel balancing&rotation", "Wheel balancing & rotation");

    /** Canonical {@code services.name_en} (exact DB value). */
    private final String nameEn;

    /** Alternate Hyper {@code universalServiceId} spellings for this {@code name_en}. */
    private final String[] extraHyperIds;

    HyperServiceMapping(String nameEn, String... extraHyperIds) {
        this.nameEn = nameEn;
        this.extraHyperIds = extraHyperIds;
    }

    public String getNameEn() {
        return nameEn;
    }

    /**
     * True when {@code hyperUniversalServiceId} maps to the given {@code percentageNameEn} row.
     */
    public static boolean matches(String hyperUniversalServiceId, String percentageNameEn) {
        if (hyperUniversalServiceId == null || hyperUniversalServiceId.isBlank())

```

```

        || percentageNameEn == null || percentageNameEn.isBlank()) {
            return false;
        }
        String hyperId = hyperUniversalServiceId.trim();
        for (HyperServiceMapping mapping : values()) {
            if (!mapping.nameEn.equalsIgnoreCase(percentageNameEn)) {
                continue;
            }
            if (mapping.nameEn.equalsIgnoreCase(hyperId)) {
                return true;
            }
            for (String id : mapping.extraHyperIds) {
                if (id.equalsIgnoreCase(hyperId)) {
                    return true;
                }
            }
        }
        return false;
    }
}

/**
 * Resolve a Hyper id to {@code name_en} when the mapping is unambiguous.
 * Returns empty when unknown or when multiple rows could match the same Hyper id.
 */
public static Optional<String> toNameEn(String hyperUniversalServiceId) {
    if (hyperUniversalServiceId == null || hyperUniversalServiceId.isBlank()) {
        return Optional.empty();
    }
    String hyperId = hyperUniversalServiceId.trim();
    String matched = null;
    for (HyperServiceMapping mapping : values()) {
        if (!mapping.hyperIdMatches(hyperId)) {
            continue;
        }
        if (matched != null && !matched.equalsIgnoreCase(mapping.nameEn)) {
            return Optional.empty();
        }
        matched = mapping.nameEn;
    }
    return Optional.ofNullable(matched);
}

private boolean hyperIdMatches(String hyperId) {
    if (nameEn.equalsIgnoreCase(hyperId)) {
        return true;
    }
    for (String id : extraHyperIds) {
        if (id.equalsIgnoreCase(hyperId)) {
            return true;
        }
    }
    return false;
}
}

```

Generated for Product Owner review from the live `carland_service` source. If the enum changes in git, regenerate this PDF from `docs/generate_hyper_service_mapping_pdf.py`.